Assignment 2 Question 2: Interprocess Communication (IPC)

1. Introduction

Interprocess Communication (IPC) allows two or more processes to exchange information while executing. Processes normally operate independently, so the operating system provides mechanisms that allow them to communicate and coordinate their activities.

This assignment demonstrates IPC using unnamed pipes in the C programming language. Two processes are created: a parent process and a child process. They exchange messages in both directions using two pipes.

2. Objectives

The objectives of this implementation are to:

- Create two processes using `fork()`.
- Establish communication between processes using unnamed pipes.
- Send data from the parent process to the child process.
- Send a response from the child process to the parent process.
- Demonstrate basic process coordination and synchronization.

3. Programming Language and Tools

Item	Description
Programming language	C
Operating system	Parrot OS (Linux)
Compiler	GCC
IPC mechanism	Unnamed pipes
System calls	`pipe()`, `fork()`, `read()`, `write()`, `close()`, `wait()`
Editor	Kate

4. Implementation

4.1 IPC Mechanism Used

The program uses unnamed pipes to enable communication between the parent and child processes.

A pipe is a communication channel that allows one process to send data to another. A standard pipe supports communication in one direction, so two pipes are used to enable two-way communication.

- Pipe 1: Transfers data from the parent to the child.
- Pipe 2: Transfers data from the child to the parent.

4.2 Main System Calls

System call	Purpose
`pipe()`	Creates a communication pipe.
`fork()`	Creates a child process.
`write()`	Sends data through a pipe.
`read()`	Receives data from a pipe.
`close()`	Closes unused pipe ends.
`wait()`	Makes the parent wait for the child process to finish.

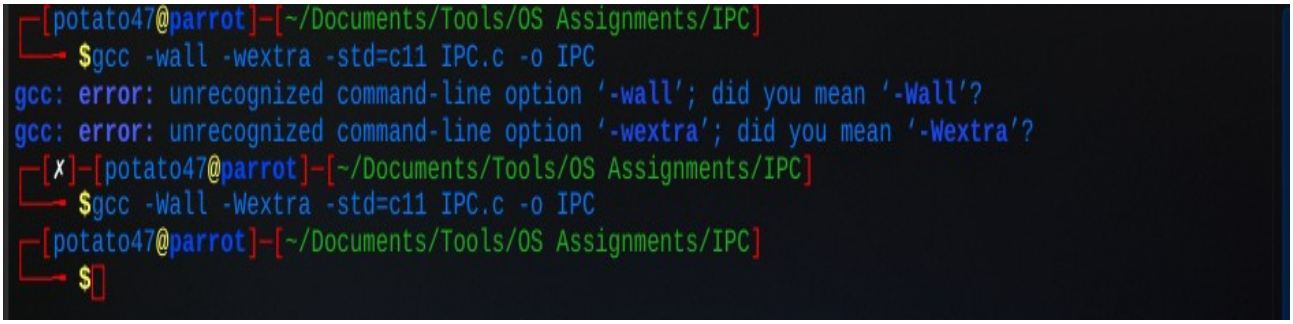
4.3 Program Execution

The program follows these steps:

1. Two pipes are created.
2. `fork()` creates a child process.
3. The parent sends a message through the first pipe.
4. The child reads the message.
5. The child sends a response through the second pipe.
6. The parent reads the response.
7. The parent waits for the child process to finish.

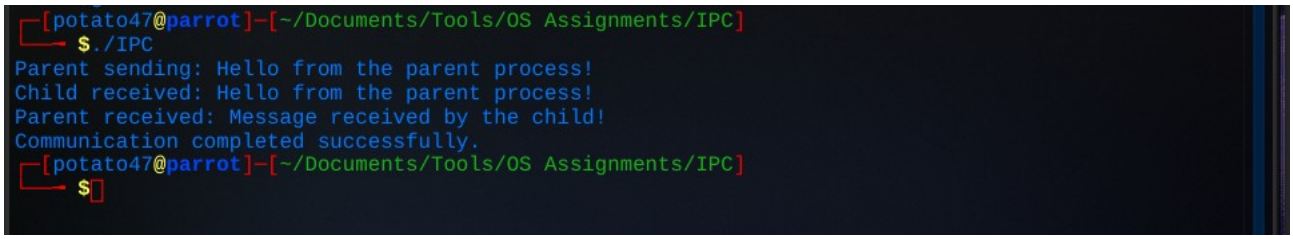
4.4 Compilation and Execution

The program was compiled using GCC, and there was a typo in the command during compilation



5. Testing and Results

The program is tested by running the parent and child processes and observing whether messages are successfully exchanged in both directions.



6. Synchronization and Error Handling

Synchronization is important because processes may execute at different speeds.

This implementation uses two basic mechanisms:

- Blocking read: A process calling `read()` waits when no data is available in the pipe, allowing it to receive data when the other process sends it.
- Process synchronization: The `wait()` system call ensures that the parent waits for the child process to terminate.

The program should also check whether system calls such as `pipe()`, `fork()`, `read()` and `write()` succeed. If a system call fails, an error should be reported rather than assuming communication was successful.

Closing unused pipe ends helps prevent unnecessary resource usage and allows the pipe to behave correctly.

7. Discussion

The implementation demonstrates how the operating system enables communication between independent processes through IPC mechanisms.

The parent and child processes exchange information without needing to access each other's memory directly. The operating system manages the pipe buffers and coordinates access to the communication channels.

Using two pipes makes bidirectional communication possible. The implementation also demonstrates the importance of synchronization when multiple processes are executing concurrently.

One limitation is that unnamed pipes are generally used for communication between related processes, such as a parent and its child. Other IPC mechanisms, such as named pipes, shared memory and message queues, can support different communication requirements.

8. Conclusion

The assignment successfully demonstrates the use of unnamed pipes for interprocess communication in C. Two processes are created using `fork()` and exchange messages through separate pipes.

The implementation illustrates how the operating system provides communication and synchronization mechanisms that allow processes to cooperate during execution.

9. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts. Wiley.
2. Linux Manual Pages: `pipe(2)`, `fork(2)`, `read(2)`, `write(2)` and `wait(2)`. https://man7.org/linux/man-pages/
3. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. Operating Systems: Three Easy Pieces. https://pages.cs.wisc.edu/~remzi/OSTEP/